

CSA 1717-ARTIFICIAL INTELLIGENCE

LAB EXPERIMENTS

Abinaya Sri J(192524429)

AIM

To solve the 8-Queens problem using the Backtracking algorithm so that no two queens attack each other on an 8×8 chessboard.

OBJECTIVE

To place eight queens on an 8×8 chessboard without conflicts. To understand the working of the Backtracking algorithm for solving constraint satisfaction problems.

ALGORITHM

Step 1: Start with an empty 8×8 chessboard.

Step 2: Begin placing the first queen in Row 0.

Step 3: For the current row, check each column one by one.

Step 4: Before placing a queen, verify that:

- No queen is present in the same column.
- No queen is present in the left diagonal.
- No queen is present in the right diagonal.

Step 5: If the position is safe, place the queen.

Step 6: Move to the next row and repeat Steps 3–5.

Step 7: If no safe position is available in a row:

- Remove the previously placed queen (Backtracking).
- Try the next column in the previous row.

Step 8: Continue until all 8 queens are placed successfully.

Step 9: Display the final chessboard.

SOURCE CODE

N = 8

```

# Create an empty chessboard
board = [[0] * N for _ in range(N)]

# Function to check whether a queen can be placed
def safe(row, col):

    # Check same column
    for i in range(row):
        if board[i][col] == 1:
            return False

    # Check left diagonal
    i = row - 1
    j = col - 1
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1

    # Check right diagonal
    i = row - 1
    j = col + 1
    while i >= 0 and j < N:
        if board[i][j] == 1:
            return False
        i -= 1
        j += 1

```

```
return True
```

```
# Backtracking function
```

```
def queen(row):
```

```
    # If all queens are placed
```

```
    if row == N:
```

```
        return True
```

```
    # Try every column
```

```
    for col in range(N):
```

```
        # Check safe position
```

```
        if safe(row, col):
```

```
            # Place queen
```

```
            board[row][col] = 1
```

```
            # Move to next row
```

```
            if queen(row + 1):
```

```
                return True
```

```
            # Backtrack
```

```
            board[row][col] = 0
```

```
return False
```

```
# Start placing queens
```

```
queen(0)
```

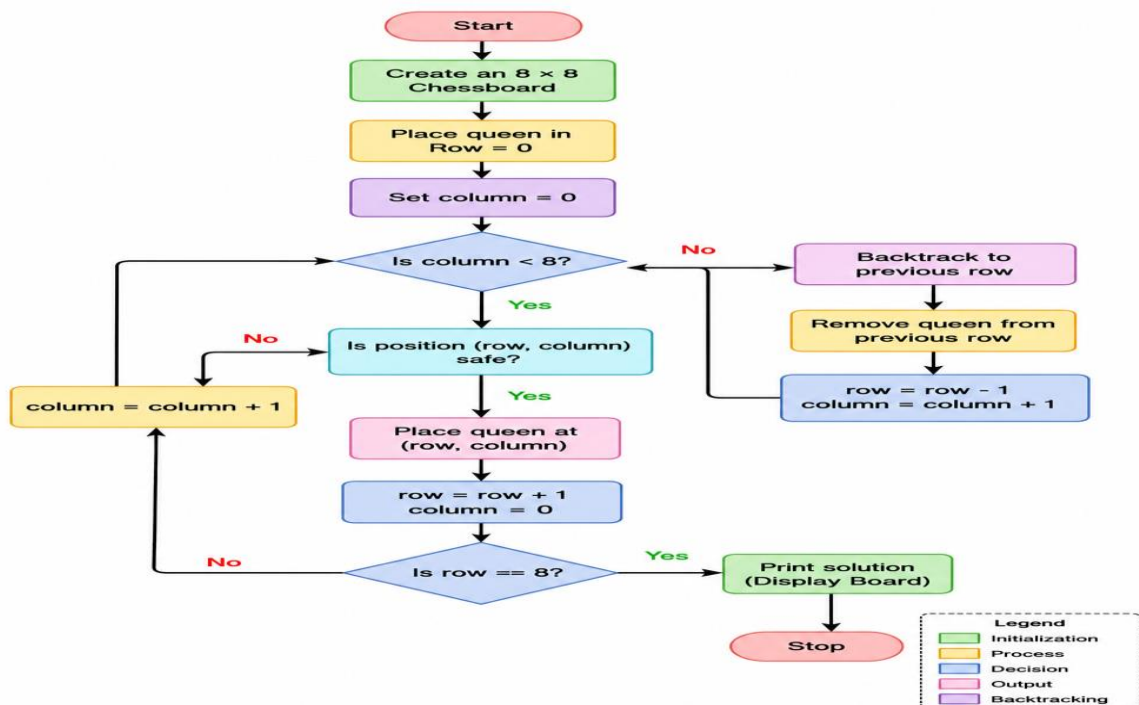
```
# Print solution
```

```
print("Solution Board:\n")
```

```
for row in board:
```

```
    print(row)
```

FLOW CHART



OUTPUT

```
Python 3.7.0 Shell: Python3 /usr/bin/python3
Solution Board:
```

```
[1, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 1, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 1]
[0, 0, 0, 0, 0, 1, 0, 0]
[0, 0, 1, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 1, 0]
[0, 1, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 1, 0, 0, 0, 0]
```

RESULT

The 8-Queens Problem was successfully solved using the Backtracking Algorithm. The program placed all eight queens on the chessboard such that no two queens attacked each other.